

# Teoria Relazionale

## Schemi e istanze

In ogni base di dati esistono:

- **Lo schema** è come un disegno o un modello fisso che descrive la struttura del database. Esso è sostanzialmente invariante nel tempo (es. le intestazioni delle tabelle come nome – cognome – indirizzo etc).
- **L'istanza** è il contenuto attualmente presente nelle tabelle (es i dati concreti, come Francesco – Giammaria – Via Antonio Scarpa) e può cambiare molto rapidamente anche solo aggiungendo/rimuovendo altre istanze.

| NOME  | COGNOME | DATAN    | LUOGON  | SCHEMA  |
|-------|---------|----------|---------|---------|
| Piero | Napoli  | 22-10-63 | Bari    | ISTANZE |
| Marco | Bianchi | 01-05-54 | Roma    |         |
| Maria | Rossi   | 09-02-68 | Milano  |         |
| Maria | Bianchi | 07-12-70 | Bari    |         |
| Paolo | Sossi   | 15-03-75 | Palermo |         |

## Linguaggi di interazione

I tipi di linguaggi principali che consentono di interagire/lavorare con i database sono due:

- **DDL (Data Definition Language):** Servono per definire la struttura del database, ad esempio per creare o modificare le tabelle.
- **DML (Data Manipulation Language):** Serve per leggere o modificare i dati.

## Modelli di Database

Nel tempo, sono stati sviluppati diversi **Modelli** per organizzare i dati all'interno dei database. Questi modelli definiscono come i dati sono collegati tra loro e come possono essere consultati. Ecco i principali:

- **Modello Reticolare:** I dati sono organizzati in una rete di record collegati tramite puntatori.
- **Modello Gerarchico:** I dati sono organizzati in modo più rigido, come un albero. Ogni dato ha un “genitore” (come una cartella che contiene altre cartelle).
- **Modello Relazionale:** Organizza i dati in tabelle di record omogenei (es. studenti, corsi, esami). Non utilizza puntatori espliciti, semplificando la rappresentazione dei dati.
- **Modello a Oggetti:** Introduce la nozione di oggetti, che contengono sia dati (attributi) che comportamenti (metodi). Questo modello è stato introdotto negli anni '80 e non ha ancora uno standard universale.

## Modello Relazionale

Il **Modello Relazionale** è un modo di organizzare i dati nei database, proposto nel 1970 da uno studioso di nome E. F. Codd. Questo modello è stato creato per rendere più facile lavorare con i dati in modo indipendente dalla struttura fisica del database (come sono memorizzati i dati "dietro le quinte"). Però ci sono voluti più di dieci anni prima che fosse implementato in sistemi reali, perché costruire database efficienti e affidabili non è semplice.

Il modello relazionale **si basa sul concetto matematico di "relazione"**. Nel contesto dei database, una relazione è molto simile a una tabella, quindi quando parliamo di "relazione" in questo ambito, in realtà stiamo parlando di tabelle, come quelle di Excel. Queste tabelle sono composte da righe e colonne, dove ogni colonna contiene un tipo specifico di dati (ad esempio nomi, cognomi, indirizzi, ecc.), e ogni riga, che viene detta tupla, contiene una serie di dati relative ad una specifica entità (come una persona, un prodotto, un esame etc)

**IMPORTANTE!** Una Relazione nel modello concettuale Entità-Relazioni viene tradotta anche con associazione perché rappresenta appunto un tipo di collegamento (relazione) concettuale tra entità diverse (ad esempio Esame mette in relazione uno Studente ed un Insegnamento) ... ma questo sarà trattato nel secondo modulo del corso.

### Definizioni

**Dominio:** l'insieme, possibilmente infinito, di appartenenza di un certo tipo di dati (es. l'insieme dei numeri interi, l'insieme dei numeri decimali, l'insieme delle stringhe di lunghezza = 20, l'insieme {0,1} e così via). Quasi sempre si usano domini predefiniti, comuni ai linguaggi di programmazione

**Relazione Matematica:** è un qualsiasi sottoinsieme del prodotto cartesiano di uno o più domini. Una relazione che è sottoinsieme del prodotto Cartesiano di k domini si dice di grado k.

Gli elementi di una relazione sono detti tuple (oppure n-uple o ennuple). Il numero di tuple di una relazione è la sua cardinalità. Ogni tupla di una relazione di grado k ha k componenti ordinate ma non c'è ordinamento tra le tuple ed esse sono tutte distinte (almeno per un valore).

Esempio:

- Supponiamo  $k = 2$

Numero dei Domini

- $D1 = \{\text{bianco, nero}\}, D2 = \{0, 1, 2\}$

Domini

$D1 \times D2 = \{(\text{bianco}, 0), (\text{bianco}, 1), (\text{bianco}, 2), (\text{nero}, 0), (\text{nero}, 1), (\text{nero}, 2)\}$

Prodotto Cartesiano

---

$\{(\text{bianco}, 0), (\text{nero}, 0), (\text{nero}, 2)\}$  è una relazione di grado 2, cardinalità 3 e con tuple (bianco, 0), (nero, 0), (nero, 2)

$\{(\text{nero}, 0), (\text{nero}, 2)\}$  è una relazione di grado 2, cardinalità 2 e con tuple (nero, 0), (nero, 2)

## Notazione

- Se  $r$  è una relazione di grado  $k$
- se  $t$  è una tupla di  $r$
- se  $i$  è un numero intero nell'insieme  $\{1, \dots, k\}$   
 $t[i]$  (oppure  $t.i$ ) indica la  $i$ -sima componente di  $t$

- esempio:

- se  $r = \{(0,a), (0,c), (1,b)\}$

- $t = (0,a)$  è una tupla di  $r$

- $t[2] = a$

- $t[1] = 0$

- $t[1,2] = (0, a)$



|   |   |
|---|---|
| 0 | a |
| 0 | c |
| 1 | b |

## Relazioni e Tabelle



### Ricapitolando:

Una relazione può essere implementata come una tabella in cui ogni riga è una tupla distinta della relazione e ogni colonna corrisponde ad una componente (valori omogenei, cioè provenienti dallo stesso dominio).

Le colonne corrispondono ai domini  $D_1, D_2, \dots, D_k$  e hanno associati dei nomi univoci all'interno della tabella, detti nomi degli attributi (descrivono il ruolo, come nome, cognome, indirizzo etc)

La coppia (nome di attributo, dominio) è chiamata attributo. L'insieme di attributi di una relazione è detto schema.

Se una relazione è denominata  $R$  e i suoi attributi hanno nomi  $A_1, A_2, \dots, A_k$ , lo schema è spesso indicato da:

$R(A_1, A_2, \dots, A_k)$

Nell'ultima definizione di modello relazionale, le componenti di una relazione sono indicate dai nomi degli attributi, anziché dalla posizione  $t[A_i]$  indica il valore dell'attributo con nome  $A_i$  della tupla  $t$

## Vincoli di Integrità

In alcuni casi le Basi di Dati vengono definite come “Scorrette” se presentano degli errori o delle incongruenze tra i dati. Esempio:

| IMPIEGATO |         |        |                |            |     |
|-----------|---------|--------|----------------|------------|-----|
| CODICE    | COGNOME | NOME   | RUOLO          | ASSUNZIONE | DIP |
| COD1      | Rossi   | Mario  | Analista       | 1795       | 01  |
| COD2      | Bianchi | Pietro | Analista       | 1990       | 05  |
| COD2      | Neri    | Paolo  | Amministratore | 1985       | 01  |

  

| DIPARTIMENTO |                 |
|--------------|-----------------|
| NUMERO       | NOME            |
| 01           | Progettazione   |
| 02           | Amministrazione |

COD2 Duplicato...

Data di Assunzione impossibile

Codice dipartimento inesistente

Oppure

| Esami | Studente | Voto | Lode | Corso |
|-------|----------|------|------|-------|
|       | 276545   | 32   |      | 01    |
|       | 276545   | 30   | si   | 02    |
|       | 787643   | 27   | si   | 03    |
|       | 739430   | 15   |      | 04    |

| Studenti | Matricola | Cognome | Nome  |
|----------|-----------|---------|-------|
|          | 276545    | Rossi   | Mario |
|          | 787643    | Neri    | Piero |
|          | 787643    | Bianchi | Luca  |

Studenti con la stessa matricola

Voti sotto il 18 e superiori al 30

27 con lode....

Tutti queste incongruenze possono essere risolte, o evitate, inserendo dei **Vincolo di integrità**, cioè delle proprietà che deve essere soddisfatta da ogni istanza della base di dati. **Un'istanza di base di dati è corretta se soddisfa tutti i vincoli di integrità associati al suo schema.**

## Vincoli INTRA-relazionali

**Vincolo di chiave primaria** (primary key): unica e mai nulla

**Vincoli di dominio** (es. voto  $\geq 18$  AND voto  $\leq 30$ )

**Vincoli di unicità** (unique)

**Vincoli di esistenza** per un certo attributo (not null)

**Espressioni sul valore di attributi** della stessa tupla (es. data\_arrivo < data\_partenza)

## Vincoli INTER-relazionali

Questi vincoli sono presenti in tabelle diverse (o relazioni). Il loro scopo principale è garantire la **consistenza** e l'**integrità** dei dati, assicurandosi che le informazioni contenute in tabelle collegate siano corrette e coerenti. Un esempio comune di vincolo inter-relazionale è la **chiave esterna** (foreign key), che permette di collegare una tabella a un'altra attraverso un campo che fa riferimento alla chiave primaria dell'altra tabella.

Ad esempio, il **vincolo di integrità referenziale** garantisce che un valore nella colonna di chiave esterna di una tabella corrisponda sempre a un valore valido nella colonna di chiave primaria della tabella a cui è collegato. Questo impedisce, ad esempio, di avere un esame associato a uno studente inesistente.

## Chiavi

Quando abbiamo una tabella (o relazione) in un database, ogni riga (o tupla) rappresenta un insieme di dati. Per poter distinguere una riga dall'altra in modo univoco, abbiamo bisogno di un metodo che permetta di identificare ciascuna riga in maniera unica. Qui entrano in gioco le **chiavi**.

**def:** Una **chiave** è un attributo o un insieme di attributi che permette di identificare **univocamente** una tupla.

Un insieme di attributi è considerato una chiave se rispetta due condizioni:

1. **Unicità:** Ogni riga deve essere unica rispetto alla chiave.

Esempio: Se "ID Studente" è una chiave, non ci saranno due studenti con lo stesso ID.

2. **Minimo insieme di attributi:** Nessun sottoinsieme della chiave può soddisfare la prima condizione. Questo significa che la chiave deve essere composta dal minor numero di attributi possibile per mantenere l'unicità.

Esempio: Se la chiave è formata da "Nome" e "Cognome", ma "ID Studente" da solo è sufficiente per garantire l'unicità, allora "ID Studente" è chiave.

A volte, una tabella (o relazione) può avere più di una possibile chiave. Queste vengono chiamate **chiavi alternative**. Tuttavia, quando dobbiamo scegliere una chiave da usare abitualmente, preferiamo quella più semplice o quella che utilizza meno attributi, chiamata **chiave primaria**, la quale deve avere alcune caratteristiche speciali:

- **Non ammette valori nulli:** Ogni riga deve avere un valore valido per la chiave primaria. Non possono esistere righe con valori nulli in una chiave primaria.
- **Deve esistere sempre una chiave:** Per garantire che non ci siano due righe uguali in una tabella, deve esserci almeno una chiave che distingue ogni riga.

## Dipendenze Funzionali

Una dipendenza funzionale stabilisce un particolare legame semantico tra due insiemi. In pratica, una dipendenza funzionale dice che:

**Se conosciamo il valore di un attributo X possiamo determinare univocamente il valore di un altro attributo Y**

Questa relazione è scritta come  $X \rightarrow Y$  e si legge **X DETERMINA Y**.

Esempio: Supponiamo di avere uno schema di relazione VOLI (CodiceVolo, Giorno, Pilota, Ora). I vincoli corrispondono alle dipendenze funzionali.

- $\text{CodiceVolo} \rightarrow \text{Ora}$  *(Se conosci il CodiceVolo, sai esattamente l'Ora di partenza)*
- $\{\text{Giorno}, \text{Pilota}, \text{Ora}\} \rightarrow \text{CodiceVolo}$  *(Se conosci il Giorno, il pilota e l'ora del volo puoi ricavare il codice del volo)*
- $\{\text{CodiceVolo}, \text{Giorno}\} \rightarrow \text{Pilota}$  *(Se conosci il CodiceVolo ed il giorno puoi ricavare chi è il pilota)*

# Progettazione di una base di dati relazionale

## Esercizio di Introduzione

Supponiamo di voler creare una base di dati contenente i seguenti dati di studenti universitari:

- **Dati anagrafici e identificativi:** nome e cognome, data, comune e provincia di nascita, matricola, codice fiscale
- **Dati curriculari per ogni esame sostenuto:** voto, data, codice, titolo e docente del corso

### Ipotesi 1

La base di dati consiste di una sola relazione con schema:

| Curriculum | Matr | CF  | Cogn    | Nome  | DataN | ComN  | ProvN | C# | TitC    | DocC  | DataE | Voto |
|------------|------|-----|---------|-------|-------|-------|-------|----|---------|-------|-------|------|
|            | 01   | ... | Rossi   | Mario | ...   | Tolfa | Roma  | 10 | Fisica  | Pippo | ...   | ...  |
|            | 02   | ... | Bianchi | Paolo | ...   | Tolfa | Roma  | 10 | Fisica  | Pippo | ...   | ...  |
|            | 01   | ... | Rossi   | Mario | ...   | Tolfa | Roma  | 20 | Chimica | Pluto | ...   | ...  |

**Problema di Ridondanza:** I dati anagrafici di uno studente sono memorizzati per ogni esame sostenuto dallo studente. I dati di un corso sono memorizzati per ogni esame sostenuto per quel corso... Questo comporta:

- **Spreco di spazio in memoria**
- **Anomalie di aggiornamento:** se cambia il docente del corso il dato deve essere modificato per ogni esame sostenuto per quel corso
- **Anomalia di inserimento:** NON posso inserire i dati anagrafici di uno studente finché non ha sostenuto almeno un esame a meno di non usare valori nulli (spreco di spazio!); idem per i corsi
- **Anomalia di cancellazione:** eliminando i dati anagrafici di uno studente potrebbero essere eliminati i dati di un corso (se lo studente è l'unico ad aver sostenuto l'esame di quel corso); idem quando elimino un corso

### Ipotesi 2 (chiaramente migliore...)

| Studente | Matr | CF      | Cogn  | Nome | Data  | Com  | Prov | Corso | C#      | Tit   | Doc |
|----------|------|---------|-------|------|-------|------|------|-------|---------|-------|-----|
| 01       | ...  | Rossi   | Mario | ...  | Tolfa | Roma | 10   |       | Fisica  | Pippo |     |
| 02       | ...  | Bianchi | Paolo | ...  | Tolfa | Roma | 20   |       | Chimica | Pluto |     |

  

| Esame | Matr | C# | Data | Voto |
|-------|------|----|------|------|
|       | 01   | 10 | ...  | ...  |
|       | 02   | 10 | ...  | ...  |
|       | 01   | 20 | ...  | ...  |

**Ridondanza:** Il fatto che un comune si trova in una certa provincia è ripetuto per ogni studente nato in quel comune

- **Anomalia di aggiornamento** - Se un comune cambia provincia (in seguito alla creazione di una nuova Provincia) devono essere modificate più tuple
- **Anomalia di inserimento** - Non è possibile memorizzare il fatto che un certo comune si trova in una certa provincia se non c'è almeno uno studente nato in quel comune
- **Anomalia di cancellazione** - Se vengono eliminati i dati anagrafici di uno studente potrebbe perdersi l'informazione che un certo comune si trova in una certa provincia (se è l'unico studente nato in quel comune)

### Ipotesi 3 (perfetta)

| Studente | Matr | CF  | Cogn    | Nome  | Data | Com   |
|----------|------|-----|---------|-------|------|-------|
|          | 01   | ... | Rossi   | Mario | ...  | Tolfa |
|          | 02   | ... | Bianchi | Paolo | ...  | Tolfa |

| Comune | Com   | Prov |
|--------|-------|------|
|        | Tolfa | Roma |

| Esame | Matr | C# | Data | Voto |
|-------|------|----|------|------|
|       | 01   | 10 | ...  | ...  |
|       | 02   | 10 | ...  | ...  |
|       | 01   | 20 | ...  | ...  |

| Corso | C# | Tit     | Doc   |
|-------|----|---------|-------|
|       | 10 | Fisica  | Pippo |
|       | 20 | Chimica | Pluto |

## Uno Schema Buono

Uno schema di basi di dati è “buono” se NON presenta ridondanze, anomalie di aggiornamento, inserimento e cancellazione. Spesso la cattiva progettazione, cioè in particolare l’errore di rappresentare più concetti nella stessa relazione, deriva dalla necessità di recuperare informazioni relative non solo agli oggetti ma anche alle loro associazioni.

Es. nel caso appena visto, operazioni eseguite frequentemente potrebbero richiedere di recuperare i dati relativi a tutti gli esami sostenuti da uno studente o ai docenti dei corsi che ha frequentato, quindi memorizzare tutte queste informazioni insieme potrebbe apparire “comodo”, ma è sicuramente sbagliato....

Abbiamo visto che le operazioni dell’algebra relazionale (e quelle dei linguaggi delle basi di dati relazionali) consentono attraverso riferimenti e join di ottenere le stesse informazioni senza incorrere nei problemi che abbiamo discusso.

**Problema:** Come possono essere individuati i concetti rappresentati in una relazione (ben progettata)?

**Soluzione:** individuare i concetti rappresentati in una relazione tramite una chiave (cioè un attributo o un gruppo di attributi che determinano una particolare dipendenza funzionale che è un particolare tipo di vincolo)

## Vincoli

Un’istanza della base di dati è **legale** se soddisfa tutti i vincoli (cioè se è una rappresentazione fedele della realtà).

Un DBMS permette di:

- Definire insieme allo schema della base di dati i vincoli
- Verificare che un’istanza della base di dati sia legale
- In base a speciali vincoli predefiniti, impedire l’inserimento di tuple che violerebbero tali vincoli

Un DBMS è dotato di procedure per la verifica dei vincoli più frequenti. In questa categoria rientrano:

- **vincoli di dominio** (un voto è un intero compreso tra 18 e 30)
- **chiavi** (il numero di matricola identifica univocamente uno studente)
- **contenimento di domini** (il numero di matricola in un verbale di esame deve essere il numero di matricola di uno studente)

Per la verifica di altri tipi di vincoli (es: vincoli dinamici, vincoli che coinvolgono valori di più attributi in un’espressione matematica) può essere necessario definire opportune procedure.

Vedremo che le dipendenze funzionali definite su uno schema di relazione esprimono particolari vincoli di dipendenza tra sottoinsiemi di attributi dello schema stesso, che devono essere soddisfatti da ogni istanza.

## Schema di Relazione

Uno schema di relazione R è un insieme di attributi  $\{A_1, A_2, \dots, A_n\}$ . Notazione

$$R = A_1 A_2 \dots A_n$$

### IMPORTANTE!

Le prime lettere dell'alfabeto (tipo A, B, C) denotano singoli attributi (es.  $A = \{\text{Nome}\}$ ,  $B = \{\text{Cognome}\}$ ,  $C = \{\text{CF}\}$ )

Le ultime lettere dell'alfabeto (tipo X, Y) denotano insiemi di attributi (es.  $X = \{\text{Nome, Cognome, CF}\}$ ).

Se X e Y sono insiemi di attributi **XY** denota  $X \cup Y$

## Tupla

Dato uno schema di relazione  $R = A_1 A_2 \dots A_n$ , una tupla su R è una funzione che associa ad ogni attributo A un valore nel corrispondente dominio.

|       | NomeStud | CognomeStud | Es. sost. | Media |
|-------|----------|-------------|-----------|-------|
| $t_1$ | Paolo    | Rossi       | 2         | 26,5  |
| $t_2$ | Mario    | Bianchi     | 10        | 28,7  |

$t_1[\text{NomeStud}] = \text{Paolo}$   
 $t_1[\text{CognomeStud}] = \text{Rossi}$   
 $t_1[\text{Es. sost}] = 2$   
 $t_1[\text{MEDIA}] = 26,5$

$t_2[\text{NomeStud}] = \text{Mario}$   
 $t_2[\text{CognomeStud}] = \text{Bianchi}$   
 $t_2[\text{Es. sost}] = 10$   
 $t_2[\text{MEDIA}] = 28,7$

Se X è un sottoinsieme di R e  $t_1$  e  $t_2$  sono due tuple su R:

**$t_1$  e  $t_2$  coincidono su X se e solo se  $\forall A \in X (t_1[A] = t_2[A])$**

*se sono uguali in tutti gli attributi di X*

## Istanza di relazione

Def: Dato uno schema di relazione R, una istanza di R è un insieme di tuple su R.

Ne segue che TUTTE le “tabelle” che abbiamo visto finora negli esempi sono istanze relazione.

## Dipendenze funzionali

Dato uno schema di relazione R, una **dipendenza funzionale** su R è una coppia ordinata di sottoinsiemi non vuoti (X, Y)

- $X \rightarrow Y$  significa che X **determina funzionalmente** Y
- X = parte sinistra della dipendenza o **determinante**
- Y = parte destra della dipendenza o **dipendente**

Si faccia attenzione al fatto che le coppie sono ORDINATE!  $X \rightarrow Y$  ma non è detto il contrario.

*(es. nel caso delle macchine il modello determina la marca ma la marca non determina il modello... Poiché ogni marca ha potenzialmente più modelli).*

**NOTA:** le dipendenze funzionali non fanno altro che esprimere dei vincoli sui dati. (es. Se due tuple hanno uguale il codice fiscale (si riferiscono alla stessa persona) devono avere uguale anche la data di nascita!)

Solitamente l'insieme delle dipendenze funzionali necessarie per rendere un'istanza legale viene denotato con **F**.



Dati uno schema R e una dipendenza funzionale  $X \rightarrow Y$  su R, un'istanza r di R **soddisfa la dipendenza funzionale  $X \rightarrow Y$**  se:

$$\forall t_1, t_2 \in r \ (t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y])$$

*TUTTE le tuple uguali sugli attributi di X sono uguali anche sugli attributi di Y*

**NOTA:** ovviamente se  $t_1[X] \neq t_2[X]$  la dipendenza è soddisfatta comunque siano i valori di  $t_1[Y]$  e  $t_2[Y]$ . Poiché la premessa è falsa.

## Dipendenze Funzionali Banali

Dati uno schema di relazione R e due sottoinsiemi non vuoti X, Y di R tali che  $Y \subseteq X$  si ha che ogni istanza r di R soddisfa la dipendenza funzionale  $X \rightarrow Y$

| R | A  | B  | C  | D  |
|---|----|----|----|----|
|   | a1 | b1 | c1 | d1 |
|   | a1 | b2 | c1 | d2 |
|   | a1 | b1 | c1 | d3 |

$X = ABC$   
 $Y = AB$   
 $X \rightarrow Y$  è soddisfatta

Pertanto, se  $Y \subseteq X$  allora  $X \rightarrow Y \in F^+$ . Una tale dipendenza funzionale è detta **banale**

## Chiusura di un Insieme di Dipendenze Funzionali ( $F^+$ )

$F = \{A \rightarrow B, B \rightarrow C\}$

| A  | B  | C  | D  |
|----|----|----|----|
| a1 | b1 | c1 | d1 |
| a1 | b1 | c1 | d2 |
| a2 | b2 | c1 | d3 |

**Ogni istanza legale** (cioè ogni istanza che soddisfa sia  $A \rightarrow B$  che  $B \rightarrow C$ ) soddisfa sempre anche la dipendenza funzionale  $A \rightarrow C$  che, tuttavia, non appartiene ad F...

Ne deduciamo che: Dato uno schema di relazione R e un insieme F di dipendenze funzionali su R, ci sono delle dipendenze funzionali che NON sono in F, ma che sono comunque soddisfatte da ogni istanza legale di R.

**Def:** Dato uno schema di relazione R e un insieme F di dipendenze funzionali su R: **la chiusura di F, indicata con  $F^+$** , è l'insieme delle dipendenze funzionali soddisfatte da ogni istanza legale di R. Banalmente si ha che  $F \subseteq F^+$

## Assiomi di Armstrong

Ricordiamo che il nostro problema è calcolare l'insieme di dipendenze  $F^+$  che viene soddisfatto da ogni istanza legale di uno schema  $R$ . Partiamo da un insieme diverso, «facile» da calcolare anche se richiede tempo...

Denotiamo con  $F^A$  l'insieme di dipendenze funzionali definito nel modo seguente:

- se  $f \in F$  allora  $f \in F^A$
- se  $Y \subseteq X \subseteq R$  allora  $X \rightarrow Y \in F^A$  (**assioma della riflessività**) *(es.  $\{\text{nome, cognome}\} \rightarrow \text{nome}$ )*
- se  $X \rightarrow Y \in F^A$  allora  $XZ \rightarrow YZ \in F^A$ , per ogni  $Z \subseteq R$  (**assioma dell'aumento**) *(es.  $CF \rightarrow \text{cognome}$ . Se aggiungiamo ad entrambi i lati indirizzo avremo che  $\{CF, \text{indirizzo}\} \rightarrow \{CF, \text{indirizzo}\}$ )*
- se  $(X \rightarrow Y \in F^A)$  e  $(Y \rightarrow Z \in F^A)$  allora  $X \rightarrow Z \in F^A$  (**assioma della transitività**) *(es.  $\text{targa} \rightarrow \text{marca}$  e  $\text{marca} \rightarrow \text{modello}$ . Abbiamo quindi che  $\text{targa} \rightarrow \text{modello}$ )*

Dimostreremo che  $F^+ = F^A$ , cioè che la chiusura di un insieme di dipendenze funzionali  $F$  può essere ottenuta a partire da  $F$  applicando ricorsivamente gli assiomi della riflessività, dell'aumento e della transitività, conosciuti come assiomi di Armstrong.

Prima di procedere con la dimostrazione introduciamo ancora 3 regole conseguenza degli assiomi che consentono di derivare da dipendenze funzionali in  $F^A$  altre dipendenze funzionali in  $F^A$ .

- a) se  $X \rightarrow Y \in F^A$  e  $X \rightarrow Z \in F^A$  allora  $X \rightarrow YZ \in F^A$  (**regola dell'unione**)
- b) se  $X \rightarrow Y \in F^A$  e  $Z \subseteq Y$  allora  $X \rightarrow Z \in F^A$  (**regola della decomposizione**) *(es.  $CF \rightarrow \{\text{nome, cognome}\}$  implica che  $CF \rightarrow \text{nome}$  e  $CF \rightarrow \text{cognome}$ )*
- c) se  $X \rightarrow Y \in F^A$  e  $WY \rightarrow Z \in F^A$  allora  $WX \rightarrow Z \in F^A$  (**regola della pseudotransitività**).

### Dim di a) Regola dell'Unione:

- a) se  $X \rightarrow Y \in F^A$  e  $X \rightarrow Z \in F^A$  allora  $X \rightarrow YZ \in F^A$  (**regola dell'unione**)

Se  $X \rightarrow Y \in F^A$ , per l'assioma dell'aumento si ha  $X \rightarrow XY \in F^A$ . Analogamente, se  $X \rightarrow Z \in F^A$ , per l'assioma dell'aumento si ha  $XY \rightarrow YZ \in F^A$ . Quindi, poiché  $X \rightarrow XY \in F^A$  e  $XY \rightarrow YZ \in F^A$ , per l'assioma della transitività si ha  $X \rightarrow YZ \in F^A$ .

### Dim di b):

- b) se  $X \rightarrow Y \in F^A$  e  $Z \subseteq Y$  allora  $X \rightarrow Z \in F^A$  (**regola della decomposizione**)

Se  $Z \subseteq Y$  allora, per l'assioma della riflessività, si ha  $Y \rightarrow Z \in F^A$ . Quindi, poiché  $X \rightarrow Y \in F^A$  e  $Y \rightarrow Z \in F^A$ , per l'assioma della transitività si ha  $X \rightarrow Z \in F^A$ .

### Dim di c):

- c) se  $X \rightarrow Y \in F^A$  e  $WY \rightarrow Z \in F^A$  allora  $WX \rightarrow Z \in F^A$  (**regola della pseudotransitività**).

Se  $X \rightarrow Y \in F^A$ , per l'assioma dell'aumento si ha  $WX \rightarrow WY \in F^A$ . Quindi, poiché  $WX \rightarrow WY \in F^A$  e  $WY \rightarrow Z \in F^A$ , per l'assioma della transitività si ha  $WX \rightarrow Z \in F^A$ .

## Osservazioni

per la regola dell'unione, se  $X \rightarrow A_i \in F_A$ ,  $i=1, \dots, n$  allora  $X \rightarrow A_1, \dots, A_i \dots A_n \in F_A$  • per la regola della decomposizione, se  $X \rightarrow A_1, \dots, A_i \dots A_n \in F_A$  allora  $X \rightarrow A_i \in F_A$ ,  $i=1, \dots, n$  quindi se e solo se •  $X \rightarrow A_1, \dots, A_i \dots A_n \in F_A \Leftrightarrow X \rightarrow A_i \in F_A$ ,  $i=1, \dots, n$  e possiamo limitarci in generale a considerare le dipendenze col membro destro singleton

## Chiusura di un insieme di attributi

Siano  $R$  uno schema di relazione,  $F$  un insieme di dipendenze funzionali su  $R$  e  $X$  un sottoinsieme di  $R$ . La chiusura di  $X$  rispetto ad  $F$ , denotata con  $X^+_F$  (o semplicemente  $X^+$ , se non sorgono ambiguità) è definito nel modo seguente:

$$X^+_F = \{ A \text{ t.c. } X \rightarrow A \in F^A \}$$

In pratica fanno parte della chiusura di un insieme di attributi  $X$  tutti quelli che sono determinati funzionalmente da  $X$  eventualmente applicando gli assiomi di Armstrong. Banalmente  $X \subseteq X^+_F$  (per riflessività)

Dunque, la chiusura di  $X$  è l'insieme di attributi determinati (direttamente o indirettamente) da  $X$ . Essa, chiaramente, non è mai vuota, perché contiene almeno  $X$  stesso (per riflessività dimostrata prima).

### Def di Chiave

$X$  è una chiave quando all'interno della sua chiusura  $X^+$  su  $F$  ci sono tutti gli attributi di  $R$ . Cioè se li determina tutti.

Ma non è vero il contrario, poiché sebbene contiene tutti gli attributi NON deve contenere un'altra chiave (cioè un sottoinsieme di  $X$  che a sua volta determina tutti gli attributi).

### Esempio:

Se  $CF \rightarrow \text{Comune}$  e  $\text{Comune} \rightarrow \text{Provincia}$  avremo che la chiusura di  $CF$ , denotata con  $(CF)^+_F = \{\text{Comune}, \text{Provincia}, CF\}$

$\text{Comune}$  viene determinato direttamente,  $\text{provincia}$  viene determinato indirettamente e  $CF$  si trova lì per riflessività.

**Lemma:** Siano  $R$  uno schema di relazione ed  $F$  un insieme di dipendenze funzionali su  $R$ . Si ha che:

$$X \rightarrow Y \in F^A \text{ se e solo se } Y \subseteq X^+$$

**(Dimostrazione parte se):** Poiché  $Y \subseteq X^+$ , per ogni  $i \leq n$ , si ha che  $X \rightarrow A_i \in F_A$ . Pertanto, per la regola dell'unione,  $X \rightarrow Y \in F^A$

**(Dimostrazione parte solo se):** Poiché  $X \rightarrow Y \in F^A$ , per la regola della decomposizione si ha che, per ogni  $i$ ,  $i=1, \dots, n$ ,  $X \rightarrow A_i \in F_A$ , cioè  $A_i \in X^+$ , per ogni  $i$ ,  $i=1, \dots, n$ , e, quindi,  $Y \subseteq X^+$

## Teorema $F^A = F^+$

**Dim:** L'uguaglianza d'insiemi si dimostra con la doppia inclusione. Dobbiamo quindi mostrare che  $F^A \subseteq F^+$  e  $F^+ \subseteq F^A$

**$F^A \subseteq F^+$**  Si dimostra per **Induzione sul Numero di Applicazione degli Assiomi di Armstrong**. Vogliamo mostrare che tutto ciò che si ottiene con gli assiomi di Armstrong sta nella chiusura di  $F$ .

**Base dell'induzione:**  $i=0$ . In tal caso  $X \rightarrow Y$  è in  $F$  e quindi, banalmente,  $X \rightarrow Y$  è in  $F^+$

*(Poiché abbiamo già visto che  $F \subseteq F^+$  e dunque tutto quello che sta in  $F$  sta anche in  $F^+$ )*

**HP Induttiva:** ogni dipendenza funzionale ottenuta a partire da  $F$  applicando  $i-1$  assiomi di Armstrong è in  $F^+$

**Passo Induttivo:** Si possono verificare 3 casi, in base a all'Assioma di Armstrong applicato.

1. Se  $X \rightarrow Y$  è stata ottenuta mediante l'assioma della riflessività, si ha che  $Y \subseteq X$ . Dunque tutte le tuple di  $r$  soddisfano la dipendenza banale  $X \rightarrow Y$ .

Dunque, prese due tuple  $t_1, t_2$  dell'istanza  $r$  di  $R$  si ha che:  $t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$ .

2. Se  $X \rightarrow Y$  è stata ottenuta applicando l'assioma dell'aumento significa che, al passo  $i-1$ , abbiamo una certa dipendenza  $V \rightarrow W$  che appartiene ad  $F^A$  (per ipotesi induttiva).

A questo punto, applicando come  $i$ -esimo assioma di Armstrong l'assioma dell'aumento con  $Z$ , avremo che:

$$\frac{VZ \rightarrow WZ}{X \quad Y} \in F^+ \text{ e dunque } X \rightarrow Y \in F^+$$

3. Se  $X \rightarrow Y$  è stata ottenuta mediante l'assioma della transitività si ha che, al passo  $i-1$ , esistono due dipendenze  $X \rightarrow Z$  e  $Z \rightarrow Y$  entrambe appartenenti ad  $F^+$  (per Ipotesi Induttiva). Dunque, data **un'istanza legale**  $r \in R$ :

- SE esistono due tuple  $t_1, t_2$  tali che  $t_1[X] = t_2[X]$  allora  $t_1[Z] = t_2[Z]$   
*(poiché devono per forza rispettare la dipendenza  $X \rightarrow Z$  di  $F^+$ )*

Allo stesso modo si ha che:

- SE esistono due tuple  $t_1, t_2$  tali che  $t_1[Z] = t_2[Z]$  allora  $t_1[Y] = t_2[Y]$   
*(poiché devono per forza rispettare la dipendenza  $Z \rightarrow Y$  di  $F^+$ )*

Da cui se ne deduce che, per ogni istanza legale  $r$  di  $R$ , se esistono due tuple  $t_1, t_2$  uguali su  $X$  sono uguali anche su  $Y$ , e cioè  $X \rightarrow Y \in F^+$ .

**$F^+ \subseteq F^A$** : Si dimostra per assurdo, supponendo che esista una dipendenza funzionale  $X \rightarrow Y \in F^+$  t.c.  $X \rightarrow Y \notin F^A$ . Di fatto vogliamo mostrare che tutto ciò che sta in  $F^+$  è ottenibile tramite gli assiomi di Armstrong.

Tuttavia, per la dimostrazione, utilizzeremo una particolare istanza  $r$  di  $R$  che ha solo due tuple, uguali sugli attributi di  $X^+$  e diverse in tutti gli altri (cioè  $R-X^+$ ). Dimostrando che tale tupla è legale dimostreremo che se  $X \rightarrow Y \in F^+$  avremo necessariamente che  $X \rightarrow Y \in F^A$ .

|     | $X^+$ |   |     |   | $R-X^+$ |   |     |   |
|-----|-------|---|-----|---|---------|---|-----|---|
| $r$ | 1     | 1 | ... | 1 | 1       | 1 | ... | 1 |
|     | 1     | 1 | ... | 1 | 0       | 0 | ... | 0 |

## 1. $r$ è un'istanza legale

Sia  $V \rightarrow W$  una dipendenza funzionale in  $F$ .

- Se le due tuple di  $r$  sono diverse su  $V$ , la dipendenza è soddisfatta.
- Se le due tuple di  $r$  sono uguali su  $V$ , allora  $V \subseteq X^+$  (per come abbiamo costruito  $r$ ). MA, per il Lemma1,  $V \subseteq X^+ \Leftrightarrow X \rightarrow V \in F^A$ ;

Pertanto, per l'assioma della transitività,  $X \rightarrow V$  e  $V \rightarrow W \Rightarrow X \rightarrow W \in F^A$ . Ma, di nuovo per il Lemma1,  $X \rightarrow W \in F^A \Leftrightarrow W \subseteq X^+ \Leftrightarrow$  le due tuple sono uguali su  $W$ .

Abbiamo quindi dimostrato che, data una qualunque dipendenza  $V \rightarrow W$  di  $F$  essa è soddisfatta, poiché sia  $V$  che  $W$  sono sottoinsiemi di  $X^+$ , e dunque le due tuple sono uguali su quei valori (per come abbiamo costruito  $r$ ).

Poiché abbiamo considerato una qualunque dipendenza in  $F$ ,  $r$  le soddisfa tutte, quindi è legale.

## 2. $X \rightarrow Y \in F^+$ è per forza soddisfatta

Supponiamo per assurdo che esista una dipendenza  $X \rightarrow Y$  in  $F^+$  che non sia anche in  $F^A$ .

Abbiamo mostrato prima che  $r$  è un'istanza legale, e dunque soddisfa per forza la dipendenza  $X \rightarrow Y \in F^+$ . Ma, per come abbiamo costruito  $r$ , si ha che due tuple uguali su  $X$  sono uguali anche su  $Y \Leftrightarrow X, Y \in X^+$ .

Ma, per il Lemma1,  $Y \in X^+ \Leftrightarrow X \rightarrow Y \in F^A$ , che termina la dimostrazione.

## Commenti:

La dimostrazione di questo teorema si basa su due collegamenti MOLTO importanti:

- Il collegamento tra  $F^+$  e le istanze legali**: se una istanza è legale allora soddisfa per forza tutte le dipendenze in  $F^+$
- Il collegamento tra  $X^+$  e il sottoinsieme di dipendenze in  $F^A$  che hanno come determinante  $X$** , garantito dal Lemma1. Cioè  $Y \subseteq X^+ \Leftrightarrow X \rightarrow Y \in F^A = F^+$ .
- Calcolare  $F^A$ , e quindi  $F^+$ , richiede tempo. Ogni possibile sottoinsieme di  $R$  porta a una o più dipendenze, e i possibili sottoinsiemi di  $R$  sono  $2^{\#R} \Rightarrow$  tempo di calcolo esponenziale in  $\#R$ .

## 3NF - Terza Forma Normale

Ritorniamo al nostro esempio di base di dati che contiene le informazioni sugli studenti e sugli esami sostenuti, e ripartiamo dalla soluzione «buona» trovata alla fine. La base di dati consiste di quattro schemi di relazione:

- Studente (Matr, CF, Cogn, Nome, Data, Com)
- Corso (C#, Tit, Doc)
- Esame (Matr, C#, Data, Voto)
- Comune (Com, Prov)

### Studente

Poiché il numero di matricola identifica univocamente uno studente, ad ogni numero di matricola corrisponde:

- un solo codice fiscale (  $\text{Matr} \rightarrow \text{CF}$  )
- un solo cognome (  $\text{Matr} \rightarrow \text{Cogn}$  )
- un solo nome (  $\text{Matr} \rightarrow \text{Nome}$  ) una sola data di nascita (  $\text{Matr} \rightarrow \text{Data}$  )
- un solo comune di nascita (  $\text{Matr} \rightarrow \text{Com}$  )

Quindi un'istanza di Studente per essere legale deve soddisfare la dipendenza funzionale:

$\text{Matr} \rightarrow \text{Matr CF Cogn Nome Data Com}$

Con considerazioni analoghe abbiamo che un'istanza di Studente per essere legale deve soddisfare la dipendenza funzionale:

$\text{CF} \rightarrow \text{Matr CF Cogn Nome Data Com}$

**Pertanto sia Matricola che CF sono Chiavi di Studente.** *(Essendo entrambi singleton non ha neanche senso andare a verificare se esistono sottoschemi che determinano tutto R)*

D'altra parte possiamo osservare che ci possono essere due studenti con lo stesso cognome e nomi differenti, e quindi possiamo avere istanze di Studente che NON soddisfano la dipendenza funzionale  $\text{Cogn} \rightarrow \text{Nome}$ . E allo stesso modo per tanti altri attributi abbiamo che non soddisfano la dipendenza funzionale  $\text{Cogn} \rightarrow \text{Data}$ ,  $\text{Cogn} \rightarrow \text{Comune}$ ,  $\text{Nome} \rightarrow \text{Cogn}$  etc...

Con considerazioni analoghe possiamo concludere che le uniche dipendenze funzionali non banali che devono essere soddisfatte da un'istanza legale di Studente sono del tipo  $\text{K} \rightarrow \text{X}$ , dove K contiene una chiave (nel nostro caso Matricola o CF).

*Vedremo che questa è una prima condizione (che però va ulteriormente rifinita) per definire cos'è la Terza Forma Normale (3NF)*

### Esame

Uno studente può sostenere l'esame relativo ad un corso una sola volta; pertanto per ogni esame (identificato dallo studente e dal corso) esiste una sola data ed un solo voto. Quindi ogni istanza legale di Esame deve soddisfare la dipendenza funzionale  $\text{Matr C\#} \rightarrow \text{Data Voto}$

D'altra parte uno studente può sostenere esami in date differenti e riportare voti diversi nei vari esami. Pertanto esistono istanze di Esame che NON soddisfano una o entrambe le dipendenze funzionali  $\text{Matr} \rightarrow \text{Data}$ ,  $\text{Matr} \rightarrow \text{Voto}$

Analogamente possiamo dire che esistono istanze di Esame che NON soddisfano una o entrambe le dipendenze funzionali  $\text{C\#} \rightarrow \text{Data}$ ,  $\text{C\#} \rightarrow \text{Voto}$ .

Dunque né Matr, né C# sono chiavi prese singolarmente, MA lo sono se prese insieme. Pertanto la chiave di Esame sarà Matr, C#

*In seguito vedremo delle procedure rigorose per identificare la chiavi.*

## Per ciascuno schema di relazione

- Studente (**Matr**, CF, Cogn, Nome, Data, Com)
- Corso (**C#**, Tit, Doc)
- Esame (**Matr**, **C#**, Data, Voto)
- Comune (**Com**, Prov)

ATTENZIONE!!! Stiamo continuando ad assumere che  $COM \rightarrow Prov$ , cioè che non ci sono comuni omonimi (cosa non vera)

Le uniche dipendenze funzionali non banali che devono essere soddisfatte da ogni istanza legale sono del tipo  $K \rightarrow X$  dove  $K$  contiene una chiave (cioè  $K$  è superchiave).

## 3NF – Terza Forma Normale

Def: Dati uno schema di relazione  $R$  e un insieme di dipendenze funzionali  $F$  su  $R$ , si ha che  $R$  è in 3NF se  $\forall X \rightarrow A \in F^+$ , con  $A \notin X$  si ha che:

- $X$  è una superchiave (contiene una chiave)

OPPURE

- $A$  è primo (cioè sta in qualche chiave)

$R$  è in 3 NF se ogni dipendenza funzionale  $X \rightarrow Y \in F^+$  è tale che  $X$  è super chiave ed ogni attributo di  $Y$  (preso singolarmente) è primo.

ATTENZIONE! La condizione  $A \notin X$  è importante. Infatti, per l'assioma della riflessività, se  $A \in X$  avremo sempre  $X \rightarrow A$  in  $F^+$  e quindi in  $F^+$ , anche quando  $A$  non è primo e  $X$  non è superchiave, e quindi se considerassimo questo tipo di dipendenze nessuno schema risulterebbe in 3NF.

ATTENZIONE! Per quanto detto fino ad ora, è sbagliato scrivere  $\forall X \rightarrow A \in F$ , perché non sapremmo se e come valutare una dipendenza del tipo  $X \rightarrow AB$  (due o più attributi a destra). Se sostituisco  $\forall X \rightarrow A \in F$ , con  $\forall X \rightarrow Y \in F$ , non so come comportarmi se  $Y$  contiene sia attributi primi che non primi. Dunque è importante avere a destra singletons.

- $R=AB$
- $F=\{A \rightarrow B\}$
- La chiave è ovviamente  $A$

Vediamo se  $R$  è in 3NF ... e valutiamo le dipendenze in  $F^+$

$$F^+ = \{A \rightarrow B, AB \rightarrow AB, A \rightarrow A, B \rightarrow B \dots\}$$

AB contiene la chiave ... OK

A è chiave ... OK

**$B \rightarrow B$  dovrebbe essere una violazione perché  $B$  non è parte della chiave né contiene la chiave!!!**

Ma questo tipo di dipendenze (banali) si trova **SEMPRE** in  $F^+$   
**QUINDI NON vanno considerate**

## Forma Normale di Boyce-Codd

La 3NF non è la forma più restrittiva che si può ottenere. Ne esistono altre, tra cui la forma normale di Boyce-Codd.

**def:** Una relazione è in forma normale di Boyce-Codd (BCNF) quando in ogni dipendenza di F il determinante è una superchiave

Sostanzialmente è come la 3NF ma senza la seconda condizione che il dipendente possa essere prima. Di fatto una relazione che rispetta la forma normale di Boyce-Codd è anche in terza forma normale, ma non è vero l'opposto.

## Algoritmo per il Calcolo della Chiusura $X^+$

**begin**

$Z := X;$   $\longrightarrow Z$  deve almeno contenere  $X$  per riflessività

$S := \{A/Y \rightarrow V \in F \wedge A \in V \wedge Y \subseteq Z\}$

**while**  $S \not\subseteq Z \longrightarrow$  Usciamo quando  $S$  è proprio  $Z$ . Cioè se nell'ultima iterazione non abbiamo aggiunto nulla

**do**

**begin**

$Z := Z \cup S;$

$S := \{A/Y \rightarrow V \in F \wedge A \in V \wedge Y \subseteq Z\}$

**end**

**end**

**Dim:** Indichiamo con  $Z^i$  e  $S^i$  i valori assunti da  $Z$  ed  $S$  all' $i$ -esima iterazione del while. Da cui se ne deduce chiaramente che  $Z^0 = X$  è il valore iniziale di  $Z$  ( $X$  ne fa parte per riflessività), e che  $Z^i \subseteq Z^{i+1}$  per ogni  $i$  (poiché da  $Z$  non leviamo mai nulla!)

Vogliamo provare che  $Z^{\text{finale}} = X^+$ . E dunque che  $A \in Z^{\text{finale}} \Leftrightarrow A \in X^+$

$A \in Z^{\text{finale}} \Rightarrow A \in X^+$  cioè  $Z^{\text{finale}} \subseteq X^+$

**Base dell'Induzione:**  $i=0$ . In questo caso si ha  $Z^0 = X$ , e  $X \subseteq X^+$  sempre.

**HP Induttiva:**  $Z^{i-1} \subseteq X^+$

**Passo Induttivo:** Sia  $A$  un attributo in  $Z^i - Z^{i-1}$  (cioè è stato aggiunto al passo  $i$ -esimo). Per come funziona l'algoritmo, si ha che  $A$  è stato aggiunto in  $Z^{i-1}$  se e solo se esiste una dipendenza

$$Y \rightarrow V \in F \text{ t.c. } Y \subseteq Z^{i-1} \text{ e } A \in V$$

MA per ipotesi induttiva si ha che  $Z^{i-1} \subseteq X^+$  e dunque  $Y \subseteq X^+$ , che per il Lemma 1 significa  $X \rightarrow Y \in F^A$ .

Dunque abbiamo che  $X \rightarrow Y \rightarrow V$ , con  $A \in V$ , che implica per decomposizione,  $X \rightarrow A \Leftrightarrow A \in X^+$



$$A \in X^+ \Rightarrow A \in Z^{\text{finale}} \text{ cioè } X^+ \subseteq Z^{\text{finale}}$$

Per ipotesi si ha che  $A \in X^+$ . Ma questo è vero se e solo se, per il Lemma1,  $X \rightarrow A \in F^+ = F^+$ . Dunque, siccome la dipendenza  $X \rightarrow A$  è in  $F^+$  sarà soddisfatta per forza da ogni istanza legale.

Andiamo quindi a prendere un'apposita istanza  $r$  di  $R$ , le cui tuple sono uguali sugli attributi in  $Z^{\text{finale}}$  e diversi su tutti gli altri (cioè  $R - Z^{\text{finale}}$ ), e MOSTRIAMO CHE È LEGALE:

|     |           |   |     |   |               |   |     |   |
|-----|-----------|---|-----|---|---------------|---|-----|---|
|     | $Z^{(j)}$ |   |     |   | $R - Z^{(j)}$ |   |     |   |
| $r$ | 1         | 1 | ... | 1 | 1             | 1 | ... | 1 |
|     | 1         | 1 | ... | 1 | 0             | 0 | ... | 0 |

### 1. Mostrare che $r$ è un'istanza legale

Supponiamo per assurdo che esista una dipendenza  $V \rightarrow W$  in  $F$  che NON è soddisfatta da  $r$ . Questo avviene solo se le due tuple sono uguali su  $V$  ma diverse su  $W$ . Cioè  $V \in Z^{\text{finale}}$  e  $W \cap (R - Z^{\text{finale}}) \neq \emptyset$

(non possiamo semplicemente dire che  $W \in R - Z^{\text{finale}}$  perché  $W$  è un insieme di attributi e potremmo averne alcuni in  $Z^{\text{finale}}$  ed altri no)

MA se  $V \in Z^{\text{finale}}$  significa che al passo successivo aggiungeremo tutto quello che è determinato da  $V \Rightarrow$  l'algoritmo NON è terminato  $\Rightarrow$  contraddizione con l'ipotesi che  $Z^{\text{finale}}$  fosse quella finale  $\Rightarrow r$  è legale.

Sostanzialmente, se non mettiamo mai  $V$  in  $Z$  la dipendenza sarà sempre soddisfatta, poiché non esistono tuple uguali su  $V$ .

Se invece, a qualche iterazione  $i$ , aggiungiamo  $V$  in  $Z$  significa che al passo successivo ci metteremo tutto quello che è determinato da  $V$ , compresa  $W$  (per via della dipendenza  $V \rightarrow W$  in  $F$ ). E dunque si avrebbe che  $V, W \in Z^{\text{finale}}$  e quindi la dipendenza è soddisfatta.

### 2. Terminare la dimostrazione

Siccome  $r$  è un'istanza legale DEVE per forza rispettare la dipendenza  $X \rightarrow A$  che è in  $F^+$  (per ipotesi).

La dipendenza è rispettata  $\Leftrightarrow$  due tuple uguali su  $X$  sono uguali anche su  $A \Leftrightarrow$  sia  $X$  che  $A \in Z^{\text{finale}}$

## Decomposizione di uno Schema

def. Una decomposizione di  $R$  è una famiglia  $\rho = \{R_1, R_2, \dots, R_k\}$  di sottoinsiemi di  $R$  che ricopre  $R$ , cioè il JOIN NATURALE dei sottoschemi deve ridare lo schema originario  $R$ . I sottoschemi  $R_i$  non devono per forza essere disgiunti, possono avere attributi in comune.

Uno schema di relazione viene decomposto solitamente in 2 casi:

- Quando non è in 3NF
- Per motivi di efficienza

Se le tuple sono piccole riesco caricarne di più in memoria nella stessa operazione di lettura. Dunque, se le informazioni della tupla non vengono utilizzate quasi mai assieme conviene decomporre lo schema. Esempio:

Lo schema *Studente* potrebbe essere decomposto separando le informazioni anagrafiche (CF, Nome, Cognome, DataNascita, LuogoNascita, ecc.) da quelle accademiche, poiché raramente queste informazioni saranno utilizzate assieme (ed in quei casi è sufficiente fare un join)

## La 3NF non basta

Abbiamo già visto in precedenza che quando uno schema viene decomposto non è sufficiente che i sottoschemi siano in 3NF. Per avere una buona decomposizione è necessario che:

- I sottoschemi siano in 3NF
- La decomposizione preservi le dipendenze iniziali
- Ci sia JOIN SENZA PERDA

Esempio:

- Consideriamo lo schema  $R=(Matricola, Comune, Provincia)$  con l'insieme di dipendenze funzionali  $F=\{Matricola \rightarrow Comune, Comune \rightarrow Provincia\}$
- Lo schema non è in 3NF per la presenza in  $F^+$  della dipendenza transitiva  $Comune \rightarrow Provincia$ , dato che la chiave è evidentemente  $Matricola$  ( $Provincia$  dipende transitivamente da  $Matricola$ ).
- $R$  può essere decomposto in:
  - (1)  $R1=(Matricola, Comune)$  con  $\{Matricola \rightarrow Comune\}$
  - (1)  $R2=(Comune, Provincia)$  con  $\{Comune \rightarrow Provincia\}$} QUESTO
- Oppure
  - (2)  $R1=(Matricola, Comune)$  con  $\{Matricola \rightarrow Comune\}$
  - (2)  $R2=(Matricola, Provincia)$  con  $\{Matricola \rightarrow Provincia\}$} SDAZIATO
- Entrambi gli schemi **sono** in 3NF, tuttavia la seconda soluzione **non è soddisfacente**.
- Consideriamo le istanze **legali** degli schemi ottenuti

| R1 | Matricola | Comune | R2 | Matricola | Provincia |
|----|-----------|--------|----|-----------|-----------|
|    | 501       | Tivoli |    | 501       | Roma      |
|    | 502       | Tivoli |    | 502       | Rieti     |

- L'istanza dello schema originario  $R$  che posso ricostruire da questa (l'unico modo è di ricostruirla facendo un join naturale!) è la seguente

| R | Matricola | Comune | Provincia |
|---|-----------|--------|-----------|
|   | 501       | Tivoli | Roma      |
|   | 502       | Tivoli | Rieti     |

- **MA non è un'istanza legale** di  $R$ , in quanto **non soddisfa** la dipendenza funzionale  $Comune \rightarrow Provincia$  (che sarà pure transitiva **ma possiamo rivelare praticamente che va soddisfatta comunque!**)
- **E' evidente che c'è stato un errore di inserimento, ma non abbiamo potuto rilevarlo**

## Equivalenza tra G ed F (insiemi di dipendenze funzionali)

Per verificare che  $\rho$  sia una buona decomposizione di R sarà necessario verificare che il suo insieme di dipendenze, che chiamiamo G, preservi le dipendenze originarie di R, che chiamiamo F. Questo avviene solo se G ed F sono equivalenti.

def. Siano F e G due insiemi di dipendenze funzionali. F e G sono equivalenti ( $F \equiv G$ ) se  $F^+ = G^+$ .  
(F e G NON contengono le stesse dipendenze, ma le loro chiusure SI)

Tuttavia, calcolare la chiusura di un insieme di dipendenze richiede tempo esponenziale... e noi ne dovremmo calcolare ben 2. Come facciamo??

**Lemma:** Siano F e G due insiemi di dipendenze funzionali. Se  $F \subseteq G^+$  allora  $F^+ \subseteq G^+$

**Dim:** Applicando gli assiomi di Armstrong a G arriviamo nell'insieme  $G^A = G^+$ . Ma per ipotesi  $F \subseteq G^+$ , e dunque da G possiamo arrivare a tutto F. Ma applicando gli assiomi di Armstrong ad F possiamo arrivare a tutto  $F^A = F^+$ , e dunque abbiamo dimostrato che:

$$G \xrightarrow{A} G^+ \xrightarrow{A} F \xrightarrow{A} F^+ \quad \Rightarrow \quad G \xrightarrow{A} F^+$$

def. A questo punto possiamo dire che  $\rho$  preserva F  $\Leftrightarrow F \equiv G = \bigcup_{i=1}^k \pi_{R_i}(F)$ .

**ATTENZIONE!**  $\bigcup_{i=1}^k \pi_{R_i}(F)$  è un insieme di dipendenze funzionali dato dalla proiezione dell'insieme di dipendenze funzionali F sul sottoschema  $R_i$ . Proiettare un insieme di dipendenze F su un sottoschema  $R_i$  significa: **prendere tutte le dipendenze di  $F^+$  che hanno dipendente e determinante in  $R_i$ .**

## Verificare che $\rho$ Preservi le Dipendenze

Verificare se una decomposizione preserva un insieme di dipendenze funzionali F richiede che venga verificata l'equivalenza  $F \equiv G = \bigcup_{i=1}^k \pi_{R_i}(F)$

Ma  $F \equiv G \Leftrightarrow F^+ = G^+$  che richiede, come sempre, di verificare la doppia inclusione  $F^+ \subseteq G^+$  e  $G^+ \subseteq F^+$

- $G \subseteq F^+$ : Siccome G è una "proiezioni di F" sarà sicuramente verificato che
- Rimane quindi da verificare solo  $F \subseteq G^+$  (che poi implica  $F^+ \subseteq G^+$  per il Lemma sulle chiusure), e lo si fa tramite l'algoritmo seguente:

```
begin
Z:=X;
S:=∅;
for i:=1 to k
do   S:=S ∪ (Z ∩ Ri)+F ∩ Ri
while S ⊄ Z
do
begin
Z:=Z ∪ S;
for i:=1 to k
do S:=S ∪ (Z ∩ Ri)+F ∩ Ri
end
end
```

*Handwritten notes:*

- per ogni sottoschema (pointing to the loop)
- determinanti e dipendente contenuti in  $R_i$  (pointing to the subscript  $F \cap R_i$ )
- di cui calcoliamo la chiusura su F (pointing to the superscript  $+$ )
- prendiamo solo quelli contenuti in  $R_i$  (pointing to the subscript  $R_i$ )
- finché aggiungiamo qualcosa (pointing to the while loop)

Notiamo la relazione tra la definizione delle proiezioni di F:

$$\pi_{R_i}(F) = \{X \rightarrow Y \mid X \rightarrow Y \in F^+ \wedge XY \subseteq R_i\}$$

e il passo di aggiornamento di S dove vengono aggiunti gli attributi:

$$(Z \cap R_i)^+_{F \cap R_i}$$

Questa parte del codice prende gli attributi determinati funzionalmente dalla parte di Z che si trova in quel sottoschema, e da questi selezioniamo quelli contenuti comunque in  $R_i$

Raccogliamo poi tutto in S tramite l'unione, poiché anche G è dato dall'unione delle proiezioni sulle dipendenze dei singoli sottoschemi.

**Dim:** La parte SE non va dimostrata all'esame (e dunque non è riportata di seguito)

Per dimostrare che l'algoritmo è valido dobbiamo mostrare che  $Z^{\text{finale}} = X^+_G$ . E cioè che  $A \in Z^{\text{finale}} \Leftrightarrow A \in X^+_G$

Indichiamo con  $Z^0$  il valore iniziale di Z e con  $Z^i$  il valore di Z dopo l'i-esima iterazione del while. (chiaramente, siccome ad ogni iterazione non togliamo nulla da Z ma al massimo aggiungiamo, si ha che  $Z^i \subseteq Z^{i+1}$ ).

**Base dell'Induzione:** con  $i=0$  si ha  $Z^0 = X \subseteq X^+_G$

**HP Induttiva:**  $Z^{i-1} \subseteq X^+_G$

**Passo Induttivo:** Sia A un attributo in  $Z^i - Z^{i-1}$ , cioè un attributo che è stato aggiunto all'i-esima iterazione. Per come funziona l'algoritmo si ha che:

$$\text{esiste un indice } j \text{ t.c. } A \in (Z^{i-1} \cap R_j)^+_{F \cap R_j} \Rightarrow A \in (Z^{i-1} \cap R_j)^+_F \Rightarrow (Z^{i-1} \cap R_j) \rightarrow A \in F^+$$

Lemma1 e  $F^A = F^+$

G è una proiezione di F. MA, proiettare un insieme di dipendenze funzionali significa: prendere tutte le dipendenze di  $F^+$  che hanno dipendente e determinante nello stesso sottoschema.

Inoltre, poiché  $(Z^{i-1} \cap R_j) \rightarrow A \in F^+$ ,  $A \in R_j$  e  $Z^{i-1} \cap R_j \subseteq R_j$  si ha, per la definizione di G, che  $(Z^{i-1} \cap R_j) \rightarrow A \in G$ .

La dipendenza è in  $F^+$

Ha dipendente e determinante entrambi in  $R_j$

$$\text{MA per l'ipotesi induttiva } X \rightarrow Z^{i-1} \in G^+ \Rightarrow (X \rightarrow Z^{i-1} \cap R_j) \in G^+$$

Facendo l'intersezione con  $R_j$  al massimo leviamo qualcosa. Quindi, per la regola della decomposizione, se  $X \rightarrow Z^{i-1} \in G^+$ , allora anche  $X \rightarrow (Z^{i-1} \cap R_j)$ , che è un suo sottoinsieme, ne fa parte

$$\text{A questo punto } (X \rightarrow Z^{i-1} \cap R_j) \in G^+ \text{ e } (Z^{i-1} \cap R_j) \rightarrow A \in G \Rightarrow X \rightarrow A \in G^+ \Rightarrow A \in X^+_G$$

Trasitività

## Join Senza Perdita

def. Sia  $R$  uno schema di relazione. Una decomposizione  $\rho = \{R_1, R_2, \dots, R_k\}$  di  $R$  ha **join senza perdita** se per ogni istanza legale  $r$  di  $R$  si ha  $r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_k}(r)$ .

Se definiamo  $m_\rho(r) = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_k}(r)$  si avrà che:

a)  $r \subseteq m_\rho(r)$ : Se la decomposizione ha join senza perdita allora  $r$  ed il join naturale sono uguali ed hanno le stesse istanze. Nel caso in cui abbiamo perdita allora il join naturale ha delle istanze in più, e dunque contiene l'istanza  $r$  iniziale

b)  $\pi_{R_i}(m_\rho(r)) = \pi_{R_i}(r)$ :  $m_\rho(r)$  è dato dal join naturale di tutti i sottoschemi. Questo significa che in  $m_\rho(r)$  avremo tutte le tuple di  $R_i$  con affianco qualche attributo proveniente da altri sottoschemi. Tuttavia, quando effettuiamo la proiezione su  $R_i$ , tutti questi dati in più, spariscono, e rimangono solo le tuple iniziali di  $R_i$ . (la proiezione funziona in modo insiemistico e rimuove le tuple duplicate)

c)  $m_\rho(m_\rho(r)) = m_\rho(r)$ :

$$m_\rho(m_\rho(r)) = \pi_{R_1}(m_\rho(r)) \bowtie \pi_{R_2}(m_\rho(r)) \bowtie \dots \bowtie \pi_{R_k}(m_\rho(r))$$

Per b)

$$m_\rho(m_\rho(r)) = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_k}(r) := m_\rho(r)$$

## Algoritmo di Verifica del Join Senza Perdita

```

begin
Costruisci una tabella  $r$  nel modo seguente:


- $r$  ha  $|R|$  colonne e  $|p|$  righe
- all'incrocio dell' $i$ -esima riga e della  $j$ -esima colonna metti
- il simbolo  $a_j$  se l'attributo  $A_j \in R_i$
- il simbolo  $b_{ij}$  altrimenti


repeat
for every  $X \rightarrow Y \in F$ 
do if ci sono due tuple  $t_1$  e  $t_2$  in  $r$  tali che  $t_1[X] = t_2[X]$  e  $t_1[Y] \neq t_2[Y]$ 
then for every attribute  $A_j$  in  $Y$  do if  $t_1[A_j] = 'a_j'$  then  $t_2[A_j] := t_1[A_j]$ 
else  $t_1[A_j] := t_2[A_j]$ 
until  $r$  ha una riga con tutte  $'a'$  or  $r$  non è cambiato;
if  $r$  ha una riga con tutte  $'a'$  then  $\rho$  ha un join senza perdita
else  $\rho$  non ha un join senza perdita

```

L'algoritmo costruisce un'istanza legale che ci permette la verifica, e per costruirla fa in modo che soddisfi tutte le dipendenze in  $F$ .

**Dim:**  $\rho$  ha join senza perdita  $\Leftrightarrow$  (per ogni  $r$  legale)  $m_\rho(r) = r \Leftrightarrow r$  ha una tupla con tutte  $'a'$

La parte Se della dimostrazione non è richiesta. Quindi non è riportata di seguito.

Supponiamo per assurdo che  $\rho$  abbia join senza perdita, cioè che  $m_\rho(r) = r$  per ogni  $r$  legale, e che quando l'algoritmo termina la tabella non abbia una tupla con tutte  $'a'$ .

Poiché l'algoritmo termina solo quando non ci sono più violazioni delle dipendenze in  $F$ , la tabella  $r$  può essere interpretata come un'istanza legale di  $R$  (basta sostituire ai simboli uguali valori uguali).

Inoltre, se proiettiamo la tabella  $r$  (si proietta sulle colonne!) sugli attributi di  $R_i$  si avrà sicuramente che contiene una tupla con tutte 'a' (sono proprio le 'a' che abbiamo inserito all'inizio dell'algoritmo). E siccome l'algoritmo non modifica mai le 'a' già inserite si ha che esse rimangono fino alla fine.

E dunque abbiamo una contraddizione che dimostra quando volevamo.

## Trovare una Decomposizione

Ora affrontiamo il problema di come ottenere una decomposizione che soddisfi le nostre condizioni. Prima di tutto ci chiediamo: "è sempre possibile ottenere una tale decomposizione?"

SI: è sempre possibile, dato uno schema  $R$  su cui è definito un insieme di dipendenze funzionali  $F$ , decomporlo in modo da ottenere che:

- ogni sottoschema è 3NF
- la decomposizione preserva le dipendenze funzionali
- Abbia join senza perdita

**ATTENZIONE!** La decomposizione che si ottiene dall'algoritmo che studieremo non è l'unica possibile che soddisfi le condizioni richieste. Lo stesso algoritmo, a seconda dell'input di partenza (di cui parleremo in questa lezione) può fornire risultati diversi e tuttavia corretti.

Prima di continuare dobbiamo introdurre il concetto di «copertura minimale» di un insieme  $F$  di dipendenze funzionali. Sarà proprio una copertura minimale di  $F$  a costituire l'input dell'algoritmo di decomposizione.

## Copertura Minimale

def. Sia  $F$  un insieme di dipendenze funzionali. La sua copertura minimale è un insieme  $F^{\min}$  di dipendenze funzionali equivalente ad  $F$  (cioè hanno la stessa chiusura) tale che:

- **Per ogni dipendenza funzionale in  $F^{\min}$  il dipendente è un singleton** (ogni attributo nella parte destra è non ridondante)
- **Nessuna dipendenza in  $F^{\min}$  è ridondante.** Una dipendenza è ridondante se, togliendola da  $F$ , essa è comunque rispettata da ogni istanza legale, cioè si trova comunque in  $F^+$ .

Sostanzialmente, la copertura minimale di  $F$ , è il più piccolo insieme di dipendenze funzionali EQUIVALENTE ad  $F$  (cioè hanno la stessa chiusura). Per ottenere una copertura minimale seguiamo 3 passi:

**PASSO 1:** Ridurre i dipendenti a singleton, tramite regola della decomposizione.

**PASSO 2:** Ridurre i determinanti dove possibile.

Per ogni dipendenza  $X \rightarrow A \in F^{\min}$  verifichiamo se esiste un sottoinsieme  $Y$  di  $X$  t.c.  $A \in Y^+$ . In quel caso, la dipendenza  $X \rightarrow A \in F^{\min}$  può diventare  $Y \rightarrow A$ , sempre in  $F^{\min}$ .

### PASSO 3: Rimuovere le ridondanze.

Per ogni dipendenza  $X \rightarrow A \in F^{\min}$  ci andiamo a calcolare la chiusura di  $X$  rispetto ad  $F^{\min} - \{X \rightarrow A\}$ . Se  $A$  appartiene comunque a tale chiusura, significa che la dipendenza  $X \rightarrow A$  è ridondante, poiché è rispettata anche se viene tolta.

**Oss.** Per mostrare che i due insiemi sono equivalenti dovremmo mostrare la doppia inclusione tra  $G^+$  ed  $F^+$ . Tuttavia è facile notare che i due insiemi differiscono per una sola dipendenza, e dunque:

$$F^{\min} - \{X \rightarrow A\} = G \subseteq F \quad \Rightarrow \quad G^+ \subseteq F^+$$

Resta quindi da verificare solo  $F^+ \subseteq G^+$ , che sarà sicuramente vero se  $F \subseteq G^+$ . Proprio per questo motivo andiamo a verificare se  $A \in (X)^+_G$ , che per il Lemma 1 e  $G^A = G^+$ , significa  $X \rightarrow A \in G^+$

*(le altre dipendenze non le verifichiamo poiché sono presenti in entrambi gli insiemi, e quindi alle loro chiusure)*

## Algoritmo di Decomposizione

```

begin
S := ∅;
for every A ∈ R tale che A non è coinvolto in nessuna dipendenza funzionale in F do
    S := S ∪ {A};
if S ≠ ∅ then
    begin
        R := R - S;
        ρ := ρ ∪ {S}
    end
    if esiste una dipendenza funzionale in F che coinvolge tutti gli attributi in R
        then ρ := ρ ∪ {R}
    else for every X → A ∈ F do ρ := ρ ∪ {XA}
    end

```

*qui ci arriviamo se R è già in 3NF  
non decomponibile (dice la prof ma uso pk)*

*Per ogni dipendenza rimasta nella copertura*      *Essere uno schema oggetto con determinanti-dipendenti*

**Dim:** Dimostriamo separatamente le due proprietà della decomposizione.

### **ρ preserva F**

Sia  $G = \bigcup_{i=1}^k \pi_{R_i}(F)$ . Per come funziona l'algoritmo abbiamo che, per ogni dipendenza funzionale  $X \rightarrow A \in F$ , esiste un sottoschema di  $\rho$  che contiene  $XA$ .

Dunque, tale dipendenza di  $F$ , sarà sicuramente in  $G \Rightarrow G \supseteq F \Rightarrow G^+ \supseteq F^+$

L'inclusione  $G^+ \subseteq F^+$  è banalmente verificata in quanto, per definizione,  $G \subseteq F^+$ .

### Ogni schema di $\rho$ è in 3NF

Analizziamo i singoli casi:

1. Per come funziona l'algoritmo si ha che in  $S$  ci sono tutti quegli attributi che NON compaiono in nessuna dipendenza, e cioè non determinano e non sono determinati da nessuno. Ma questo significa che tutti attributi in  $S$  saranno per forza nella chiave del sottoschema, e dunque è in 3NF.
2. Per come funziona l'algoritmo, se tutta  $R \in \rho$  significa che c'è una dipendenza in  $F^{\min}$  che coinvolge tutti gli attributi in  $R$ . Poiché  $F^{\min}$  è una copertura minimale tale dipendenza avrà la forma  $R-A \rightarrow A$ . Inoltre, sempre perché  $F^{\min}$  è una copertura minimale, si ha che non può esistere un'altra dipendenza  $X \rightarrow$  dove  $X \subseteq R-A \Rightarrow$  dunque  $R-A$  è chiave in  $R$ .

Supponiamo ora una qualunque dipendenza  $Y \rightarrow B$  in  $F^{\min}$ . Se  $B$  è proprio  $A$  allora ricadiamo nello stesso caso di prima, e si avrebbe  $Y = R-A$  che è super chiave e rispetta la 3NF. Se invece  $B$  è diverso da  $A$ , allora  $B \in R-A$ , e quindi è primo (e rispetta la 3NF)

3. In ogni sottoschema  $XA \in \rho$ , poiché  $F^{\min}$  è copertura minimale, non ci può essere una dipendenza funzionale  $X' \rightarrow A$  in  $F$  tale che  $X' \subset X \Rightarrow X$  è chiave in  $XA$ .

Adesso, come prima, supponendo una qualsiasi dipendenza  $Y \rightarrow B$  in  $F^{\min}$ : se  $B$  è uguale ad  $A$  allora  $Y$  deve essere  $X$  (essendo  $F^{\min}$  una copertura minimale  $X$  è l'unico che determina  $A$  in quel sottoschema). Se invece  $B$  è diverso da  $A$ , allora  $B \in X$ , e quindi è primo (e rispetta la 3NF)